

Nexidia Data Exchange Framework

SDK Guide



Release: Nexidia Data Exchange Framework 12.5

Document Revision: A10

Distribution Status: Published

Publication Date: July 2025

PROPRIETARY AND CONFIDENTIAL INFORMATION

Information herein is proprietary information and trade secrets of NiCE Ltd. and/or its affiliated companies (Affiliates). This document and the information herein is the exclusive property of NiCE and its Affiliates and shall not be disclosed, in whole or in part, to any third party or utilized for any purpose other than the express purpose for which it has been provided.

IMPORTANT NOTICE

Subject always to any existing terms and conditions agreed between you and NiCE or any Affiliate with respect to the products which are the subject matter of this document, neither NiCE nor any of its Affiliates shall bear any responsibility or liability to a client or to any person or entity with respect to liability, loss or damage caused or alleged to be caused directly or indirectly by any product supplied or any reliance placed on the content of this document. This includes, but is not limited to, any interruption of service, loss of business or anticipatory profits or consequential damage resulting from the use or operation of any products supplied or the content of this document. Information in this document is subject to change without notice and does not represent a commitment on the part of NiCE or any Affiliate.

All information included in this document, such as text, graphics, photos, logos and images, is the exclusive property of NiCE or in Affiliate and is protected by United States and international copyright laws. Permission is granted to use, view and photocopy (or print) materials from this document only in connection with the products to which this document relates and subject to the terms of license applicable to such products. Any other use, copying, distribution, retransmission or modification of the information in this document without the express prior written permission of NiCE or an Affiliate is strictly prohibited. In the event of any permitted copying, redistribution or publication of copyrighted material, no changes in, or deletion of, author attribution, trademark legend or copyright notice shall be made.

Products supplied may be protected by one or more of the US patents listed at www.nice.com/Patents.

For the full list of trademarks of NiCE and its Affiliates, visit www.nice.com/NiCE-Trademarks. All other marks used are the property of their respective proprietors.

All contents of this document are: Copyright © 2025 NiCE Ltd. All rights reserved.

For assistance, contact your local supplier or nearest NiCE Customer Service Center:

EMEA Region (Europe, Middle East, Africa)
Tel: +972-9-775-3800
Fax: +972-9-775-3000
email: support@nice.com

APAC Region (Asia/Pacific)
Tel: +852-8338-9818
Fax: +852-2802-1800
email: support.apac@nice.com

North America
Tel: 1-800-663-5601
Fax: +201-356-2197
email: na_sales@nice.com

International Headquarters-Israel
Tel: +972-9-775-3100
Fax: +972-9-775-3070
email: info@nice.com

The Americas Region (North, Central, South America)
Tel: 1-800-6423-611
Fax: +720-264-4012
email: support.americas@nice.com

Israel
Tel: 09-775-3333
Fax: 09-775-3000
email: support@nice.com

France
Tel: +33-(0)1-41-38-5000
Fax: +33-(0)1-41-38-5001

Hong-Kong
Tel: +852-2598-3838
Fax: +852-2802-1800

United Kingdom
Tel: +44-8707-22-4000
Fax: +44-8707-22-4500

Germany
Tel: +49-(0)-69-97177-0
Fax: +49-(0)-69-97177-200

NiCE invites you to join the **NiCE User Group (NUG)**.

Visit the NUG Website at www.niceusergroup.org and follow the instructions.

All queries, comments, and suggestions are welcome! Please email: nicebooks@nice.com

For more information about NiCE, visit www.nice.com

CONTENTS

1: Introduction	1
Document Revision History	2
Example Use Cases	3
Best Practices	4
Project Setup	4
2: Working Tables	5
Advanced Cases	5
Compute Node Compatibility	8
3: New Action Creation	9
FileMoverInput and FileMoverOutput Usage	13
Action Configuration Options	14
Action Base - > getActionConfiguration()	17
Configure Actions – Input Type Examples	18
Configure Actions – Pivot Example	20
Add Actions to the Data Exchange Framework	22
Third Party JAR Files	25
Using IntelliJ to Create Custom Actions	26
Using IntelliJ to Create the Custom Action JAR File	29
4: RESTful API Integrations	37
5: Logging, Debugging and Exception Handling	39
Logging	39
Perform Remote Debugging	40
Exception Handling	40

[This page intentionally left blank]

Introduction

In addition to the core set of actions already available within the Data Exchange Framework, Java developers can also use the provided Eclipse SDK project to create new actions that are fully integrated with the Data Exchange Framework, and cover any gaps in functionality that may be desired for your project.

Abstractions exist that simplify the most common processing scenarios, making working table management, multi-threading, user interface changes, and other Data Exchange Framework internals transparent. Once coded, deployment to the framework is as simple as exporting a JAR file of new actions, and uploading it through the Nexidia Data Exchange Framework Admin Tool server user interface. The JAR file is automatically distributed to all connected Nexidia Data Exchange Framework Server Tool servers, and new actions are immediately available for use throughout the system.

DOCUMENT REVISION HISTORY

REVISION	SOFTWARE VERSION	DATE	DESCRIPTION
A10	12.4.2	March 2024	There were no updates in this revision.
A9	12.4.1	August 2023	There were no updates in this revision.
A8	12.4	April 2023	There were no updates in this revision.
A7	5.10	August 2022	There were no updates in this revision.
A6	5.9	May 2022	There were no updates in this revision
A5	5.8	April 2022	Upgrade Java from v8 to v11 New Action Creation.htm Using IntelliJ to Create Custom Actions.htm

EXAMPLE USE CASES

Internal to Nexidia, the Data Exchange Framework SDK is tactical tool used in the field when time lines and requirements fall outside of the regular development cycle, and commonly used custom actions would eventually become candidates for absorption into the core set of actions. These are examples of possible use cases:

- Updating audit information in a remote database at the end of each process run.
- Retrieving media files from custom appliances like Hitachi, Centera, Amazon S3, etc. based on incoming metadata.
- Extracting supplemental business data from outside APIs to join to metadata rolling through a process.
- Using Nexidia's Workbench (embedded in DEF) to manipulate media.

Alternatively, the DEF's Custom SQL action provides you with full metadata manipulation capabilities, and its Executable Wrapper action can call arbitrary executables against incoming file metadata; so a custom action may not always be the most straightforward path to filling functionality gaps.

BEST PRACTICES

Keep the following information in mind when using the Nexidia Data Exchange Framework SDK.

- All action names must end with "Action".
- All actions must be self-contained in a single file. This means, for example, that if you need to create a custom class that extends Runnable for custom multi-threading, that class must be an inner class of the action.
- These are examples of properly formatted action names:
 - JSONFormatterAction
 - PrependAction
 - SecureDeleteAction

PROJECT SETUP

- Copy the CustomDEFActions.zip file from the original installation folder, and extract it to the desired location.
- In Eclipse, select File > Import > General > Existing Projects Into Workspace, select your extracted project's top-level folder, and click "Finish".
- You can create new actions in an arbitrary package structure, and export them for consumption by the Data Exchange Framework.

Working Tables

Each process in the Data Exchange Framework uses a working table named for its process group, in `working_table_{process_group_name}` form. This table contains important internal tracking and other data that the DEF uses in processing rows moving through the system, including the flow of files themselves, so it's not something to consider manipulating directly lightly. It can often be useful, however, when you need to access some metadata rolling through that isn't immediately available, for example, in the `FileToProcess` objects that most SDK abstraction templates use to hand off work to your custom code.

You can access the `WorkingDatabaseManager` class from within your custom action by calling the `getWorkingDatabaseManager()`, and use it for more direct access to the data flowing through the process' working table.

The included javadoc includes `WorkingDatabaseManager` and linked `WorkingDatabaseDAO` method descriptions for getting at public methods. You can use these, for example, for pulling specific metadata for individual `FileToProcess` file keys using `getMetadata(...)`, rename a file key with `renameKey(...)`, manually update the disposition of a column with `updateDisposition(...)`, set the segment flag on a row using `setSegmentFlag(...)` (making the row invisible downstream, and flagging it for migration to the rollover table at the end of your process run), and much more.

ADVANCED CASES

There may be advanced cases where you want to skip an action's automatic gathering of files to process and have your action get those directly, in which case extending `ActionComponent`, overriding `gatherFilesToProcess()` to return false, and then using one of the `WorkingDatabaseManager`'s `createInputDispositionAndClause(...)` methods to query for rows in your action's current disposition would be the most direct and performant approach. For example:

```
// get a connection to the working database
try (Connection connection = getWorkingDatabaseManager().getConnection()) {
    try (ResultSet rs = connection.createStatement().executeQuery("select \"FileName\",
        \"OtherMetadata1\", \"OtherMetadata2\" from \"\" + workingTableName + \" \" +
        getWorkingDatabaseManager().createInputDispositionAndClause(connection, getInputDispositions(),
        getOutputDisposition(), true))) {
        while (rs.next()) {
            ...
        }
    }
}
```

The `createInputDispositionAndClause(...)` method has several overloaded options to meet different needs, but the most comprehensive one accepts the following options:

CONNECTION	OPTION
Connection connection	The database connection to use, preferably as noted above.
List<String> inputDispositions	<code>getInputDispositions()</code> from inside your action to get the correct values to use here.
String outputDisposition	<code>getOutputDisposition()</code> from inside your action to get the correct value to use here.
boolean needsWhere	Set 'true' to have the statement start with a 'where', 'false' to have it start with an 'and' (useful when you already have some 'where' clause written that this should be appended to).
String workingTableAlias	With complex statements, it may be necessary for you to disambiguate between two table's column values by using an alias for the working table. You can specify that alias here, then use it in your update statement.
boolean ignoreSegmentFlag	Set 'true' to ignore the <code>SegmentFlag</code> column's value during the check; 'false' to weed out any results where <code>SegmentFlag == 1</code> .
Timestamp dateLimit	The variable 'dateLimit' is already available in the action, and should be used directly here. When non-null, the working table query will be adjusted to only look back as far as this date limit. When using the available variable in the action, you're telling the query to honor the self-reconciliation max days setting at the Process level.

CONNECTION	OPTION
boolean orderByCreateDate	Set 'true' to order results by CreateDate; 'false' to leave ordering to the database.
String orderOfProcessing	Valid values here can be "Default", "Ascending", or "Descending" (Strings), and are only applied if orderByCreateDate is 'true'.
UUID computeNodeUUID	A call to getMyComputeNodeUUID() should be enough here. See the "Compute Node Compatibility" section for more.

Having your action reach into the working table's internals requires a solid understanding of what you're doing, and very thorough testing of all process flow scenarios, to avoid damage to the working table.

As an alternative to having your action manipulate working table data directly, you can use the Custom SQL action in the process designer. You can configure this action to run against the working table and perform any SQL function supported by the backing database. The same caveats to direct working table manipulation apply here, but the advantage of using a simpler SDK abstraction for your main work and then updating via SQL in process design may be a clearer approach.

COMPUTE NODE COMPATIBILITY

Compute Nodes in DEF provide horizontal scalability for actions that process from metadata. By default, that behavior is fairly automatic, and actions don't need to think much more about it.

To participate, each process must have Compute Node integration turned on in its configuration. Still, when set, actions that don't process from metadata are expected to populate the working table with work to spread out. Those who process from metadata get work assigned via a "for_node" column in the working table.

If your action is configured to not process from metadata because it handles its own working table integration, it can implement the `com.nexidia.neat.processor.components.base.ComputeNodeIntegrator` marker interface to let the process engine know that compute node integration will be handled by the action itself (see the `computeNodeUUID` note in the Working Tables section above).

New Action Creation

Important

Java Runtime Version 11.0.14 is required to create custom actions.

The Data Exchange Framework SDK provides several abstractions out of the box that can be extended to simplify action component development. In addition to the methods noted below which are required to be implemented by subclasses, new actions usually include some configurable options that are presented to the user in the Nexidia Data Exchange Framework Admin Tool user interface. Details on adding those are included in the “Configure Actions” section.

- **ActionBaseFileProcessor** (for an example, see the PrependerAction class in the SDK ‘examples’ folder) — Created to address the most common use case, this abstract class can be extended to easily create an action that takes file-based inputs and outputs (file system, CIFS, or SFTP), integrates with the Data Exchange Framework working table, and multi-threads the processing of those files, leaving implementation to the developer. The following abstract method implementations cover the full integration with the Data Exchange Framework processing framework:
 - **getUIFriendlyName()** — The name of the action that is presented to the user in the Nexidia Data Exchange Framework Admin Tool user interface.
 - **getDescription()** — A description of the action, also to appear in the Nexidia Data Exchange Framework Admin Tool user interface.
 - **validateSave()** and **validateRuntime()**:
 - The validateSave() method performs validation in the Nexidia Data Exchange Framework Admin Tool user interface when the action is saved. These checks cover validation of user input.

-
- The `validateRuntime()` method performs validation in the Nexidia Data Exchange Framework Admin Tool user interface when the process itself is published, and also prior to execution of the process. Validations performed here can address structural and other NEST-related environmental concerns (additional checks around input and output types or counts, connectivity, etc.).
 - **`processFile(...)`** — Each file moving through the action is pushed through this method, where custom file analysis, transformations, etc. can be performed. Data Exchange Framework `FileMoverInput` and `FileMoverOutput` objects are passed in along with a reference to the file, offering a means to seamlessly access files over SFTP, CIFS, or directly from the file system (more details on using these objects below). For working table integration, returning a null value will simply track the fact of the action having processed the file in the working table, but adding additional metadata tied to each file is as convenient as returning a Map of name/value pairs. In that case, each name will translate to a column in the working table with a normalized name (created dynamically if it doesn't exist), and the values will be tied to the row representing the file.
 - **`configure()` and `cleanUp()`** - The `configure()` method allows you to initialize any common variables or do any other preparatory steps prior to action execution. The `cleanUp()` method allows for cleanup of any items initialized in the `configure()` call after action execution.
 - **`ActionBaseFileProcessorWithQuantum`** (for an example, see the `PrependerActionWithQuantum` class in the SDK 'examples' folder) — The process quantum is already implemented in the Data Exchange Framework core File Mover, File Discovery, and Database-to-CSV actions, and, as entry points into a process, those should cover most use cases, leaving new actions to consume what the core actions have already isolated for processing. In the event that this approach doesn't cover a new action's use cases, integration with the quantum can happen seamlessly by extending this class. All selection of files based on the quantum happens behind the scenes, but one new method is required:
 - **`isApplyMaxQuantumBound()`** — Returning false here works the way most quantum windows will, advancing in even increments from the quantum lower date bound at the specified interval. If there's a possibility that the window might pass the files available for processing, then returning true here means that the quantum's next start window will never exceed the timestamp of the most recent file it touched during its run.

- **ActionBaseMultiThreader** (for an example, see the `PrependerABMHAction` class in the SDK 'examples' folder) — This abstraction is a little more flexible than the file-based ones above, extending input and output types to include Data Exchange Framework database resources at either end, or to even require no resource inputs and/or inputs at all. It is still integrated with the Data Exchange Framework working table, though the javadoc notes explain an approach to overriding the initial configuration that will disconnect the action from the working table altogether. In addition to methods listed for the `ActionBaseFileProcessor` above, and excluding `processFile(...)`, subclasses must implement these new methods:
 - **getInputTypes()** — Return one of `InputType.FILES`, `InputType.DATABASE`, or `InputType.NONE`.
 - **getOutputTypes()** — Return one of `OutputType.FILES`, `OutputType.DATABASE`, or `OutputType.NONE`.
 - **getItemsToProcess()** — Return a list of arbitrary objects to process. These objects' only requirement is the implementation of the `WorkingTableIntegrator` interface, and each will ultimately be passed through the next method for processing in much the same way that files are passed through the `processFile(...)` method of the `ActionBaseFileProcessor`.
 - **processItem(...)** — Each item gathered from `getItemsToProcess(...)` will be pushed through this method, where custom analysis, transformations, etc. can be performed. For working table integration, returning a null value will simply track the fact of the action having processed the file in the working table, but adding additional metadata tied to each file is as convenient as returning a Map of name/value pairs. In that case, each name will translate to a column in the working table with a normalized name (created dynamically if it doesn't exist), and the values will be tied to the row representing the item.
 - **configure() and cleanUp()** - The `configure()` method allows you to initialize any common variables or do any other preparatory steps prior to action execution. The `cleanUp()` method allows for cleanup of any items initialized in the `configure()` call after action execution.

- **ActionBase** — All of the abstractions listed above are extensions of this intermediate abstraction, which can also be subclassed for a bit more flexibility. This class provides no automatic integration with the Data Exchange Framework working table or multi- threading models, however, so those are left to the developer. Multi-threading can be accomplished without using the Data Exchange Framework hooks using standard Java practices, but working table access is most easily accomplished by instantiating a FileToProcess object with an appropriate working table file key (use the same key for both parameters in the constructor), and calling its updateDisposition() method for simple action tracking, or its ingestMetadata(...) method with a map of name/value pairs to be added to the working table metadata alongside the disposition tracking. The primary difference in this class from those listed above is that in addition to the same getUIFriendlyName(), getDescription(), validateSave(), and validateRuntime() methods, two additional ones are required:
 - **getActionConfiguration()** — Returns a configuration object that will be used to set up the action's basic context.
 - **execute()** — Executes the action after validation.
 - **incrementTotalDone()** - In order to integrate with the process builder's "Live View," and to insure proper reporting, every item successfully processed should be followed by a call to incrementTotalDone().
- **ActionComponent** — This abstract class is the root of all other abstractions, and is the one that the Data Exchange Framework core classes (prior to the development of the SDK) have extended. It provides a great deal more control over the action's running environment, but requires considerably greater understanding of the Data Exchange Framework internals than the abstractions provided above. It is not recommended for most use cases, and no examples are provided, but for those actions that fall outside of the use cases covered above, Nexidia can provide implementation guidance.

FILEMOVERINPUT AND FILEMOVEROUTPUT USAGE

The FileMoverInput and FileMoverOutput classes provide a flexible approach to handling files transparently across SFTP, CIFS, and direct file system access, and should be preferred when flexible, file-based resource inputs and outputs are required.

These classes are automatically provided in the ActionBaseFileMover, which handles their life cycles (so the correct usage is handled automatically there). For other cases, keep these additional points in mind:

- All actions have access to `getInputs()` and `getOutputs()` methods, and these return a collection of Data Exchange Framework ResourceComponent objects. These ResourceComponents can be `NxFileSystemResource`, `NxDatabaseResource`, or `NxSFTPResource` objects, and these provide access to all of the configuration fields seen for those resources in the Nexidia Data Exchange Framework Admin Tool user interface. By themselves, though, the two file-based ones provide no file-handling capabilities, which is where the FileMovers come in. With a call to `FileMoverFactory.getFileMoverInput(input)` or `FileMoverFactory.getFileMoverOutput(output)`, a corresponding FileMover will be returned with which one can get file listings, get input and output streams for individual files within those resources, test for existence, rename, etc. (see the javadocs for more details).
- The best practice for using these is to declare the FileMoverInput/Output variables as null outside of a try/catch block, to instantiate them from the FileMoverFactory within that block, and then, in the block's "finally" block, if the objects are not null, to call the `finish()` method on them.
- Code must also be structured to account for failures by calling `reset()` on them if, for example, `IOExceptions` or other problems occur. `Reset()` is really only required when using SFTP resources, but the best practice is to make file-based implementations transparent, so making the call will guarantee no issues if SFTP resources aren't explicitly disallowed (in `validateRuntime()`, e.g.).

ACTION CONFIGURATION OPTIONS

Many of the action configuration options are inherited from a common set of actions, and those are automatically applied to new actions depending on their input and output types, working table integration, etc.

For fields specific to the action's own processing, though, new ones can be easily added to the user interface using the Data Exchange Framework `FormFieldMetadata` annotation on any developer-defined variable, along with appropriate getter and setter methods for each.

Once that's done, those options are presented to the user in the Nexidia Data Exchange Framework Admin Tool user interface when configuring the custom action, and can be referenced any time during action execution.

As an example, the following code inside of an action creates a new text field in the user interface that displays in the Nexidia Data Exchange Framework Admin Tool user interface as "Text to prepend." Input is required, and the "helpText" content is presented when you hover over the context-sensitive help icon next to the field.

```
@FormFieldMetadata(id = "prependertext", text = "Text to prepend", input =
FormFieldMetadata.TEXT_INPUT_TYPE, required = true, helpText = "Text entered here will be
prepended to each file name in the output path. For validation testing purposes, this
must not contain the letter 'z'.")
protected String prependertext = "";

public String getPrependertext() {
    return prependertext;
}

public void setPrependertext(String prependertext) {
    this.prependertext = prependertext;
}
```

The `FormFieldMetadata` annotation supports many options that may be useful for development. The full set is included here for reference:

- `id` — The name given to the UI element. This is transparent to the user, and is internal to the Data Exchange Framework, so should be something clear, simple, and following the same conventions as Java variable naming (no spaces or special characters, etc.). In most cases, it is clearest to simply use the variable name in this attribute.
- `text` — The text to be displayed to the user in the action component configuration dialog.
- `input` — The input type. Supported values for the SDK are:

- `TEXT_INPUT_TYPE` — for a basic text input.
- `INT_INPUT_TYPE` — for an integer input component. Can also be accompanied by `minVal` and `maxVal` attributes.
- `SELECT_INPUT_TYPE` — for a select list. Should always be accompanied by a comma-delimited data attribute that will contain all options in the select list, and may also be accompanied by a `defaultSelection` attribute to pre-select one of those options. Note that `defaultSelection` accepts a `String`, but that string should contain a number referencing the position of the item to default-select, starting at “0”.
- `PASSWORD_INPUT_TYPE` — for a masked text input field.
- `TEXTAREA_INPUT_TYPE` — for a larger text input area without word wrapping.
- `TEXTAREA_WRAP_INPUT_TYPE` — for a larger text input area with wrap wrapping.
- `TIME_INPUT_TYPE` — for a time-only input component.
- `DATETIME_INPUT_TYPE` — for a date-based input component.
- `required` — Indicates whether a value is required for the field or not. Note that `INT`, `SELECT`, and `TIME/DATE`-based types are always automatically flagged as required, since their presence always requires a value to be returned.
- `helpText` — Help text to appear on hover over the context-sensitive help icon next to the field.
- `advanced` — Boolean flag indicating whether the field should appear in the “advanced” section of component configuration. Advanced options are not shown to users automatically, but appear when they click “Show Advanced” in the action component configuration.
- `pivot` - A more advanced option that controls the visibility of some fields based on the selections in a `SELECT` input type. For a simple example, the following pivot would show the “flattenRecursion” field in the UI when value of the `SELECT` type that it’s attached to is set to “true,” and it will hide that field if it’s set to “false”: `pivot = "false;;true:flattenRecursion"`. The pivot can become quite complex, but the basic idea is that it applies only to `SELECT` form field types, and it must follow the format of semicolon- delimiting the select lists options, and then, for each one of those, a colon followed by a comma-delimited list of fields that are to be shown when that item is selected. All fields tied to other, non-selected options in the pivot definition will be hidden.

All common options' purposes can be found in the integrated Help section for various actions, and while most are transparent to SDK developers, one exception worth noting is the "File filter pattern" option. If using your own calls to a FileMoverInput to retrieve directory listings, the first parameter for is an array of Strings that's used for filtering. Correct values there can be easily pulled from the common configuration option from within your action using this syntax: `parseFileFilterPattern(getFileFilterPattern())`.

Action Base - > getActionConfiguration()

The action configuration is key to the ActionBase abstraction's ability to remove some of the ActionComponent abstraction's complexity. All implementations of the method will look essentially the same:

```
/** *
 * @see com.nexidia.neat.processor.components.base.ActionBase#getActionConfiguration()
 */
@Override
protected ActionComponentConfiguration getActionConfiguration() {
    ActionComponentConfiguration config = new ActionComponentConfiguration();
    // the types of inputs (NONE, FILES, or DATABASE)
    config.setInputType(InputType.NONE);
    // the types of outputs (NONE, FILES, or DATABASE)
    config.setOutputType(OutputType.FILES);
    // whether the action will get and set working table dispositions or not
    config.setWorkingTableIntegrated(true);
    // if the action will be multi-threaded or not (determines UI options)
    config.setMultiThreaded(true);
    // if the action will implement a quantum or not
    config.setUsingQuantum(false);
    return config;
}
```

Configure Actions – Input Type Examples

Seven input examples are shown below:

- TEXT_INPUT_TYPE


```
@FormFieldMetaData(id = "fileFilterPattern", text = "File filter", input =
FormFieldMetaData.TEXT_INPUT_TYPE, defaultVal = "*.csv", required = true, orderIndex = 50)
protected String fileFilterPattern = "*.csv";
```
- INT_INPUT_TYPE


```
@FormFieldMetaData(id = "batchCommitSize", text = "Batch execute size", input =
FormFieldMetaData.INT_INPUT_TYPE, advanced = true, minVal = 100, defaultVal = "1000",
helpText = "Help text goes here.")
private Integer batchCommitSize = 1000;
```
- SELECT_INPUT_TYPE


```
@FormFieldMetaData(id = "importOnlyRequiredData", text = "Import only required columns",
input = FormFieldMetaData.SELECT_INPUT_TYPE, data = "false,true", defaultSelection = "0",
helpText = "Help text goes here.")
private Boolean importOnlyRequiredData = false;
@FormFieldMetaData(id = "existingDataHandling", text = "How to handle existing data", input =
FormFieldMetaData.SELECT_INPUT_TYPE, data = "UPDATE,DELETE,ERROR", defaultSelection =
"0", helpText = "Help text goes here.")
private String existingDataHandling = "UPDATE";
```
- PASSWORD_INPUT_TYPE


```
@FormFieldMetaData(id = "password", text = "Password", input =
FormFieldMetaData.PASSWORD_INPUT_TYPE, required = true, helpText = "Help text goes
here.")
private String password;
```
- TEXTAREA_INPUT_TYPE


```
@FormFieldMetaData(id = "sql", text = "SQL", input = FormFieldMetaData.TEXTAREA_INPUT_
TYPE, required = true, orderIndex = 1, helpText = "Help text goes here.")
private String sql;
```
- TEXTAREA_WRAP_INPUT_TYPE

```
@FormFieldMetaData(id = "columnNameRemapping", text = "Column header re-mapping",  
input = FormFieldMetaData.TEXTAREA_WRAP_INPUT_TYPE, helpText = "Help text goes here.",  
advanced = true)
```

```
private String columnNameRemapping;
```

- TIME_INPUT_TYPE

```
@FormFieldMetaData(id = "blackoutStarttime", text = "Blackout Start Time", input =  
FormFieldMetaData.TIME_INPUT_TYPE, helpText = "The starting HH24:MM:SS of any given  
day's blackout window, during which no process will be allowed to launch. Note that an  
already-running process will continue execution during this window.")
```

```
private Timestamp blackoutStarttime;
```

Configure Actions – Pivot Example

In this pivot example, the pivot field is “processFromMetadata”. If “processFromMetadata” is “false”, then the “recursive” and “applyQuantum” fields will be displayed. If “processFromMetadata” is “true”, then the “filePathColumnName” and “treatMissingFilesAsErrors” fields will be displayed.

Important

The “applyQuantum” field is also a pivot field.

```
@FormFieldMetaData(id = "processFromMetadata", text = "Process files from metadata", input =
FormFieldMetaData.SELECT_INPUT_TYPE, data = "false,true", defaultSelection = "0", common =
true, required = true, pivot =
```

```
"false:recursive,applyQuantum>true:filePathColumnName,treatMissingFilesAsErrors",
orderIndex = 25, helpText = "Help text goes here.")
```

```
private Boolean processFromMetadata;
```

```
@FormFieldMetaData(id = "recursive", text = "Search directories recursively", input =
FormFieldMetaData.SELECT_INPUT_TYPE, data = "false,true", defaultSelection = "1", pivot =
"false;;true:flattenRecursion")
```

```
private Boolean recursive = true;
```

```
@FormFieldMetaData(id = "applyQuantum", text = "Apply process quantum", input =
FormFieldMetaData.SELECT_INPUT_TYPE, data = "false,true", defaultSelection = "0", pivot =
"false;;true:applyMaxQuantumBound", helpText = "Help text goes here.
```

```
private Boolean applyQuantum = false;
```

```
@FormFieldMetaData(id = "filePathColumnName", text = "File path column name", input =
FormFieldMetaData.TEXT_INPUT_TYPE, required = true, defaultVal = "FileName", helpText = "The
name of the metadata column in the process group's working table that will be used to determine
files to move ('FileName' is the default, as it's the working table's unique identifier for each row,
and represents a relative path to files moving through).")
```

```
private String filePathColumnName;
```



```
@FormFieldMetaData(id = "treatMissingFilesAsErrors", text = "Treat missing files as errors",  
input = FormFieldMetaData.SELECT_INPUT_TYPE, data = "false,true", defaultSelection = "0",  
helpText = "By default, missing files are considered acceptable, as systems largely import  
metadata and then wait for files to arrive, in which case we don't need to capture and track errors  
when they're not aligned. Change to 'true' here, though, if missing files should be considered  
errors to be logged and tracked against the action's failure threshold.")  
private Boolean treatMissingFilesAsErrors = false;
```

ADD ACTIONS TO THE DATA EXCHANGE FRAMEWORK

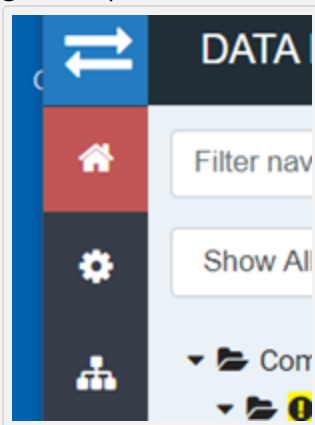
Packages within JAR files can be optionally sealed, so all classes defined in that package must be archived in the same JAR file. You might want to seal a package, for example, to ensure version consistency among the classes in your software.

If your action required the use of an additional library JAR file that's not already part of the DEF, it's possible to add it manually to NEAT and NEST server {DEF_INSTALL}\lib\ folders and restarting the DEF before deploying the custom action. Note, however, that this requires special consideration during DEF upgrade scenarios, as the Cumulative Update tool of the DEF replaces the lib\ folder contents with the latest when it's run; if the upgrade does not include your custom action JAR, it must be added again by hand.

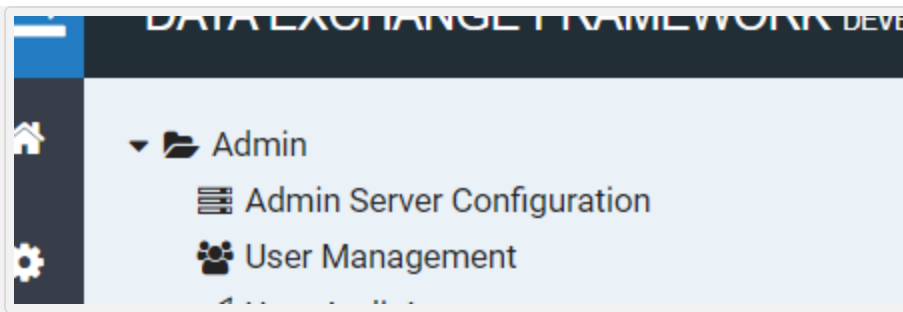
Once a custom action.JAR file has been created, the JAR file needs to be deployed to the running DEF instance. The DEF Admin Server UI will deploy the custom action JAR file.

→ To deploy the new custom action JAR file:

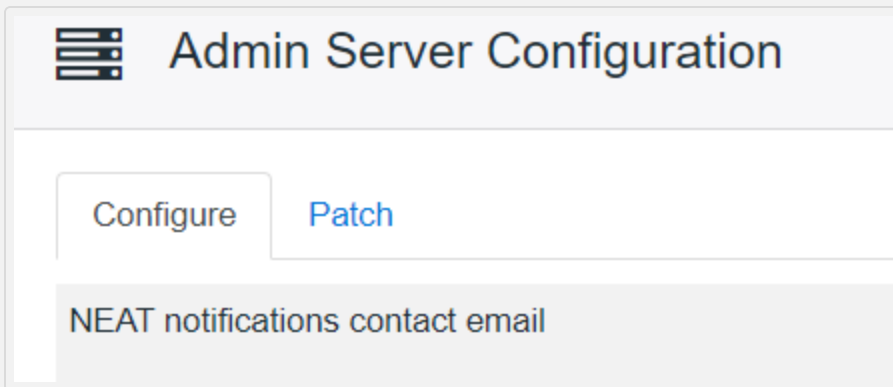
1. Login to the DEF Server Admin application.
2. Click on the **Settings** icon located in the left hand navigation bar. The **Settings** icon is the gear shaped icon.



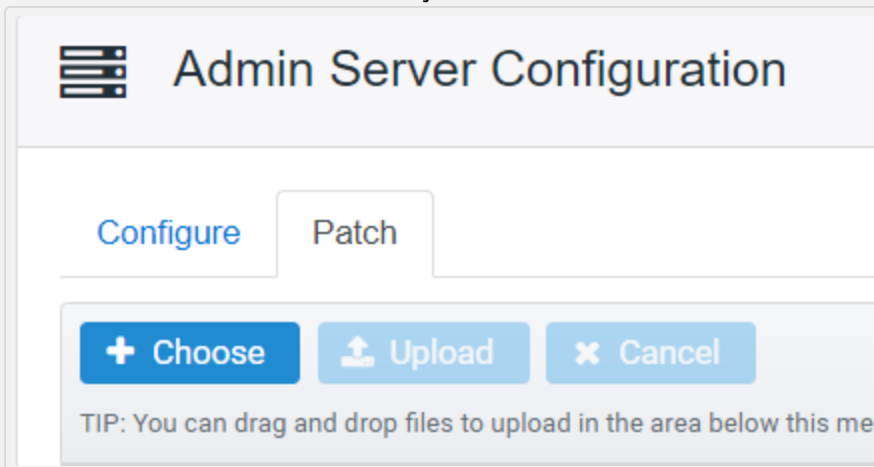
3. Click on the **Admin Server Configuration** option.



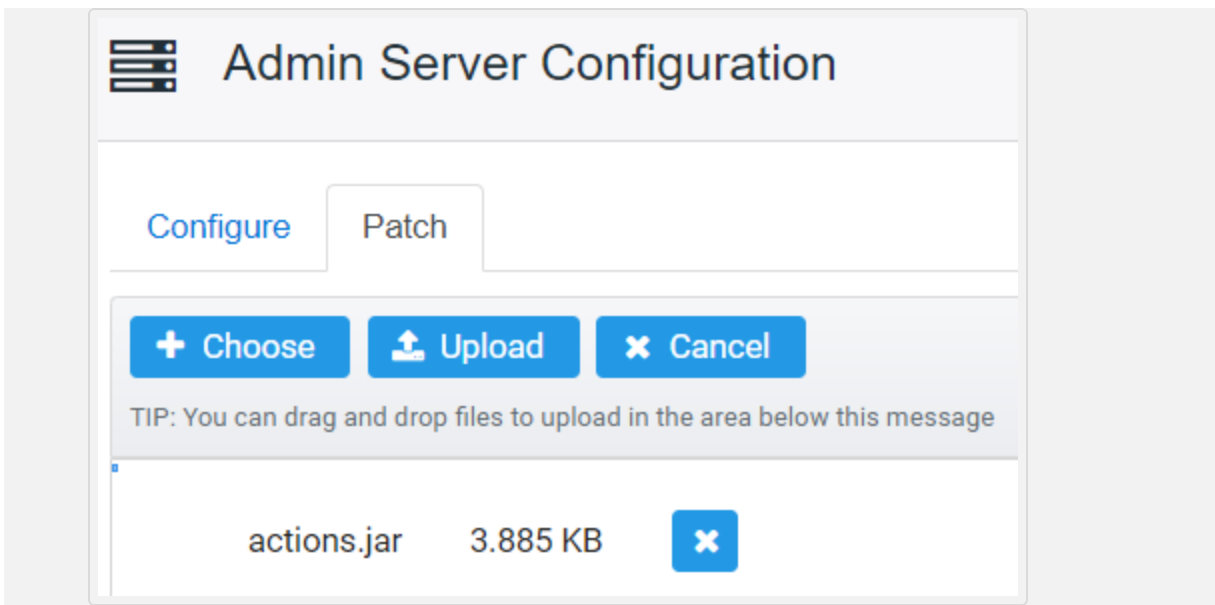
- Click on the **Patch** tab.



- Click **Choose** to select the actions.jar file.

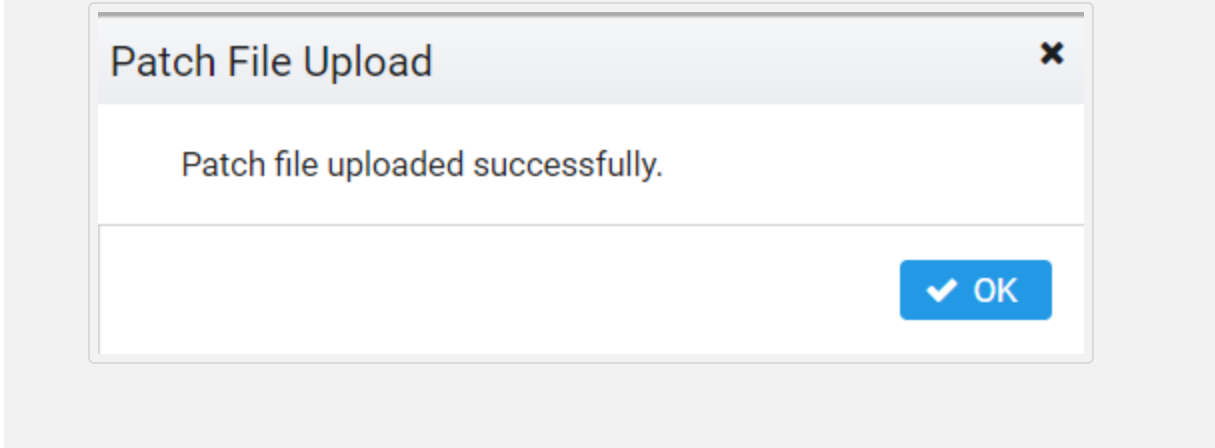


- Click **Upload** to upload the selected actions.jar file to the DEF Admin Server.



The screenshot shows the 'Admin Server Configuration' window with the 'Patch' tab selected. Below the tabs are three buttons: '+ Choose', 'Upload', and 'Cancel'. A tip message states: 'TIP: You can drag and drop files to upload in the area below this message'. Below the tip, a file named 'actions.jar' (3.885 KB) is shown with a close button (X).

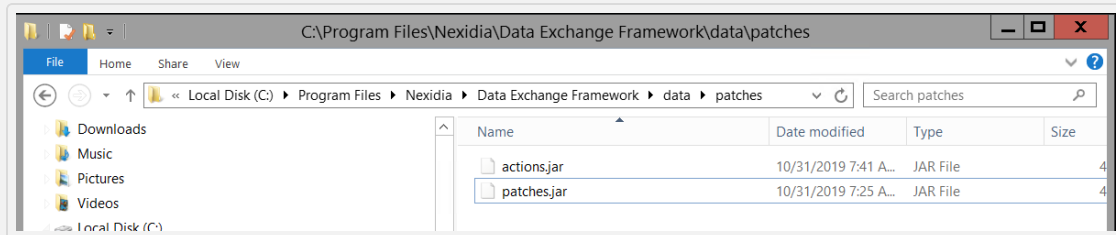
7. Click **OK** once the actions.jar file has been successfully uploaded. Note that the upload is very fast.



The screenshot shows a 'Patch File Upload' dialog box with a close button (X) in the top right corner. The message 'Patch file uploaded successfully.' is displayed in the center. At the bottom right, there is a blue button with a checkmark and the text 'OK'.

➔ **To un-deploy a previously deployed custom action JAR file:**

1. Delete the uploaded actions.jar file from the DEF Admin Server data/patches folder.



2. Restart the DEF NEAT Server.

Third Party JAR Files

The DEF SDK is delivered with two hundred and twenty three third- party JAR files. These JAR files are used by the DEF to implement base DEF action functionality, For example, File Mover action, Email Parse Text action, CSV to Database action, etc.

These JAR files can also be used to implement custom DEF actions. Third party JAR file functionality includes:

- Apache Commons including Collections, Compress, CSV, IO, Math, Pool
- Apache CXF – SOAP, XML/HTTP • Apache HTTP – Client, Core, Mime, Services
- Apache Tika – Text Processing • AWS S3, SQS, Cognito • CIFS – CIFS/SMB protocols
- Jackson – JSON processing
- Java Mail
- JDBC – SQL Server, Postgres
- Nexidia Workbench API

For the complete list of delivered third party JAR files, please see the <Custom Action Project>/libs folder for a full listing of all the third-party JAR files delivered with the DEF SDK.

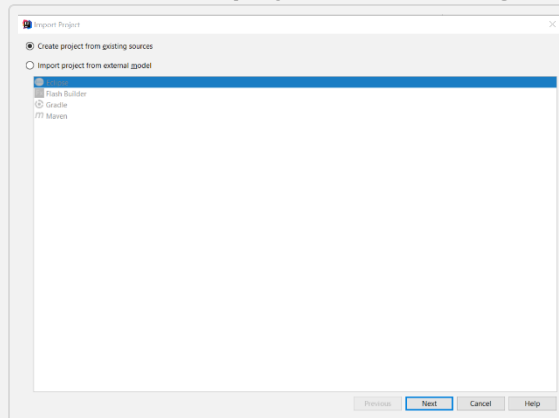
It's also possible to add your own JAR files to DEF deployments, but this is discouraged, and available libraries should be strongly preferred. See [Add Actions](#) for notes on including your own external JAR files.

Using IntelliJ to Create Custom Actions

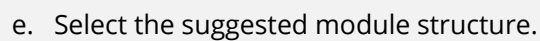
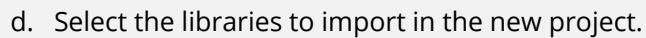
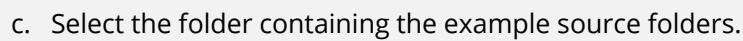
You are not limited to using the Eclipse IDE to create DEF custom actions. IntelliJ can also be used to create custom actions.

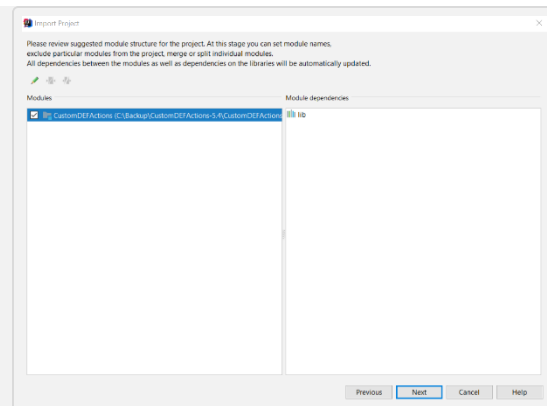
→ To import the DEF Custom Action SDK project into IntelliJ:

1. Unzip the DEF Custom Action SDK.
2. Click on **File**→ **New**→ **Project** From Existing Sources.
3. Select the file or directory to import (should be the folder containing the unzip DEF Custom Action SDK).
4. Click through the Import Project wizard.
 - a. Select the **Create project from existing sources** option.

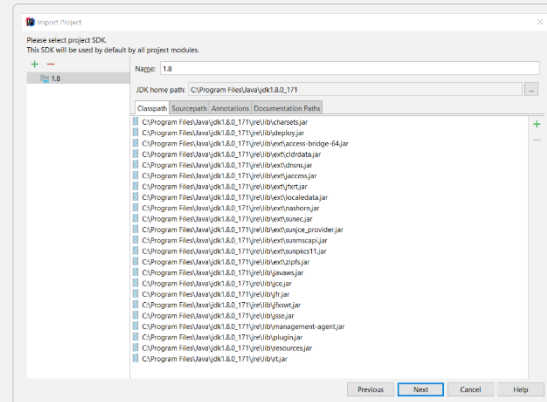


- b. Specify a project name for the new custom action.

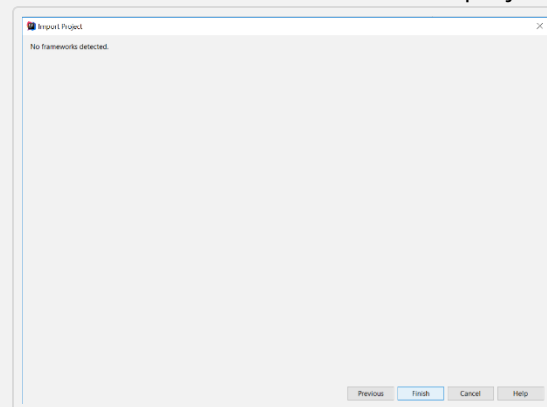




f. Select the project SDK (Java Version 11.0.14).



g. Click the **Finish** button to create the project.

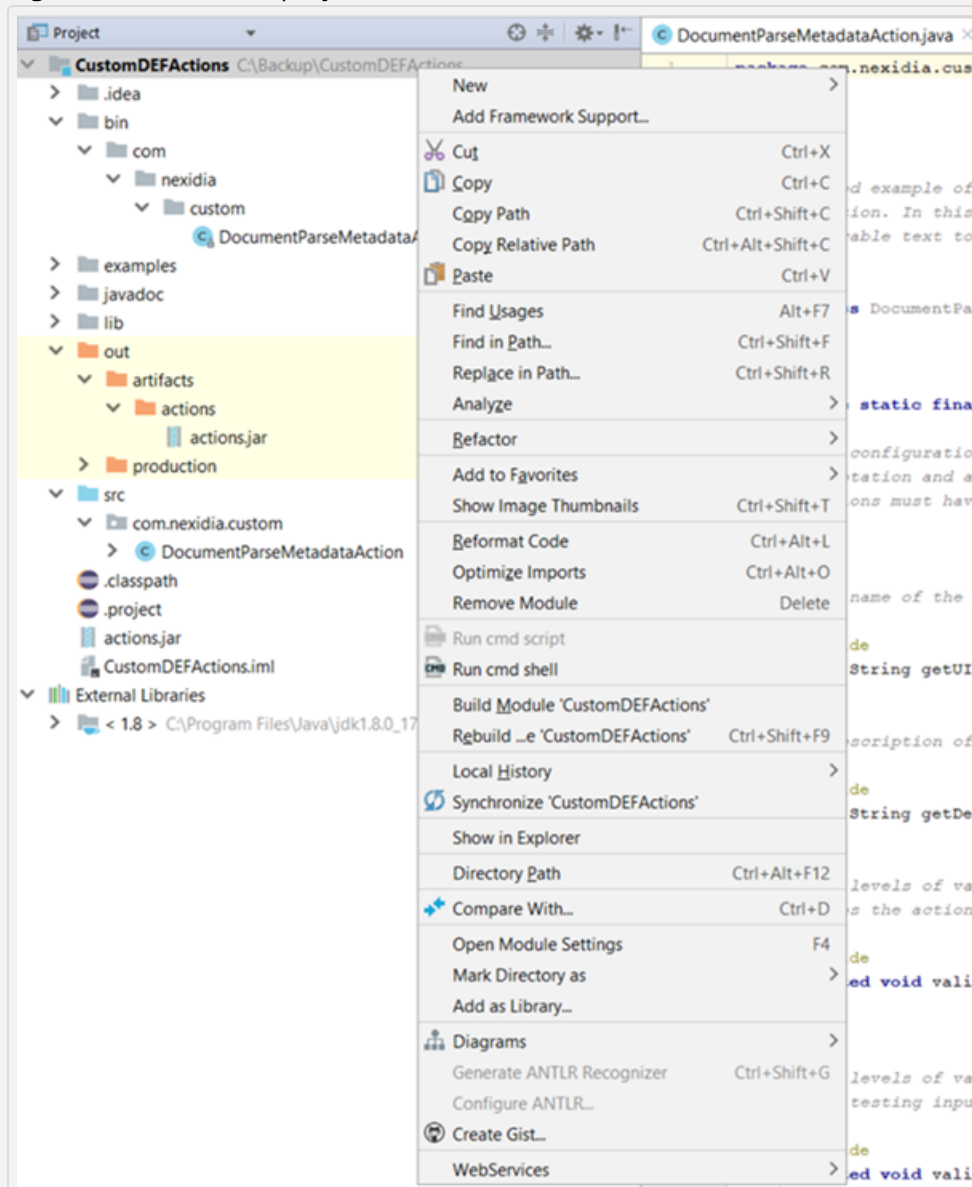


Using IntelliJ to Create the Custom Action JAR File

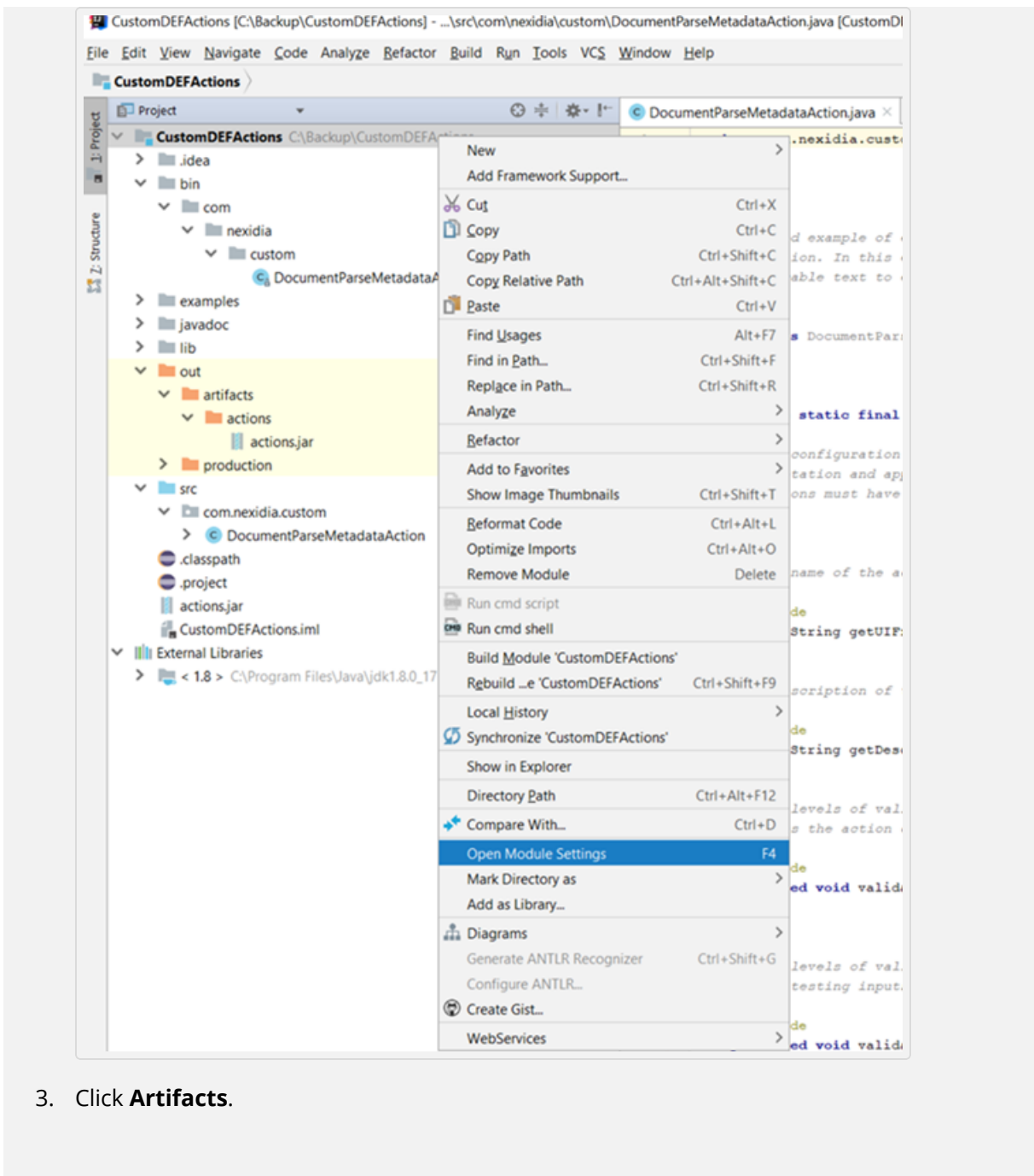
You can use IntelliJ to create the custom action JAR file: actions.jar.

→ To create the custom action JAR file in IntelliJ:

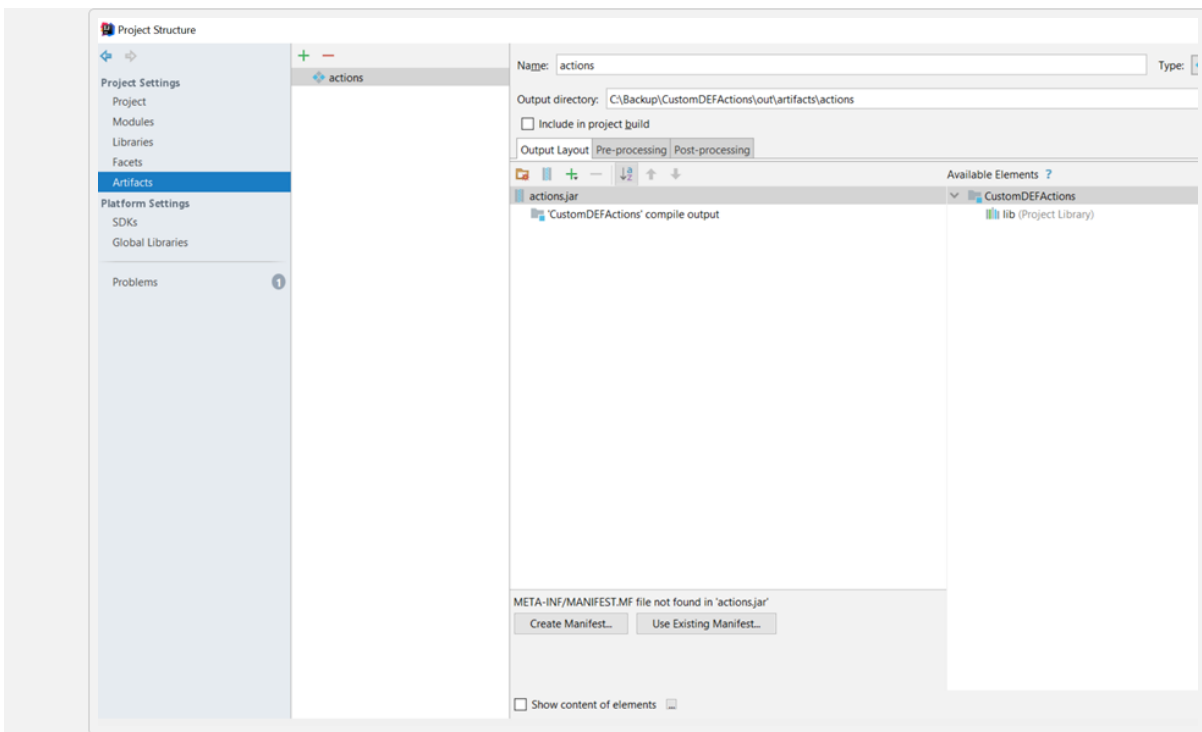
1. Right click on the SDK project.



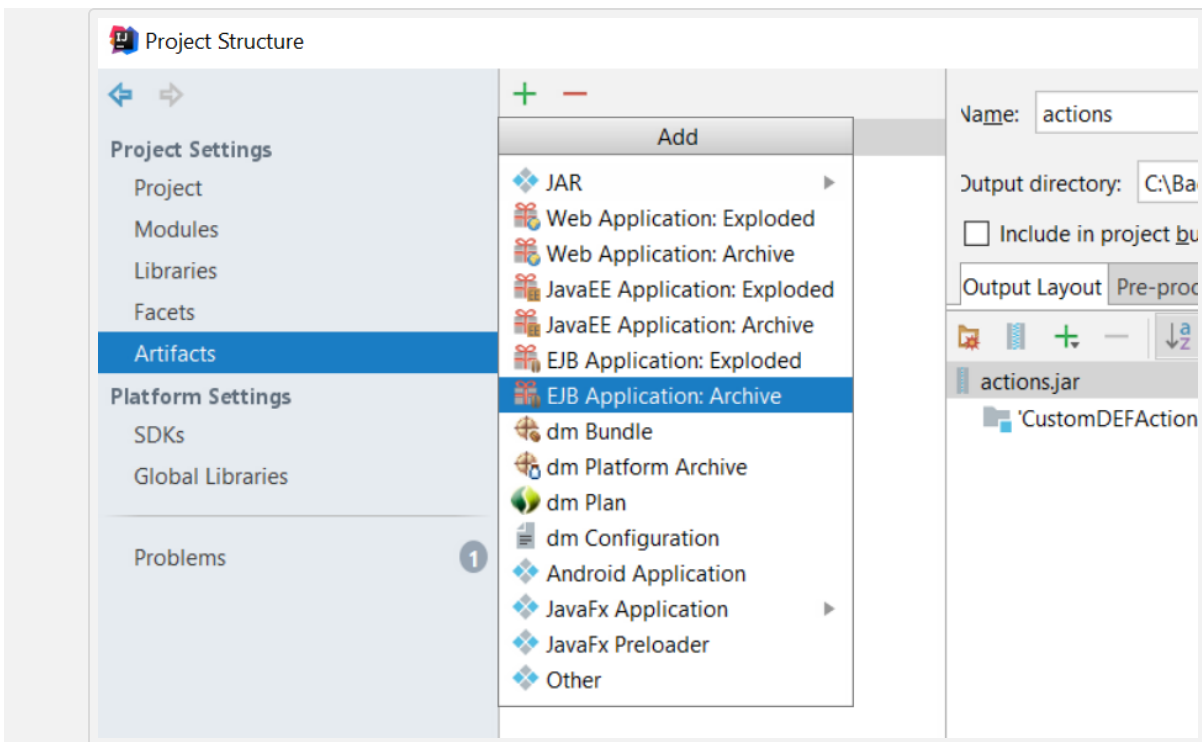
2. Click on **Open Module Settings**.



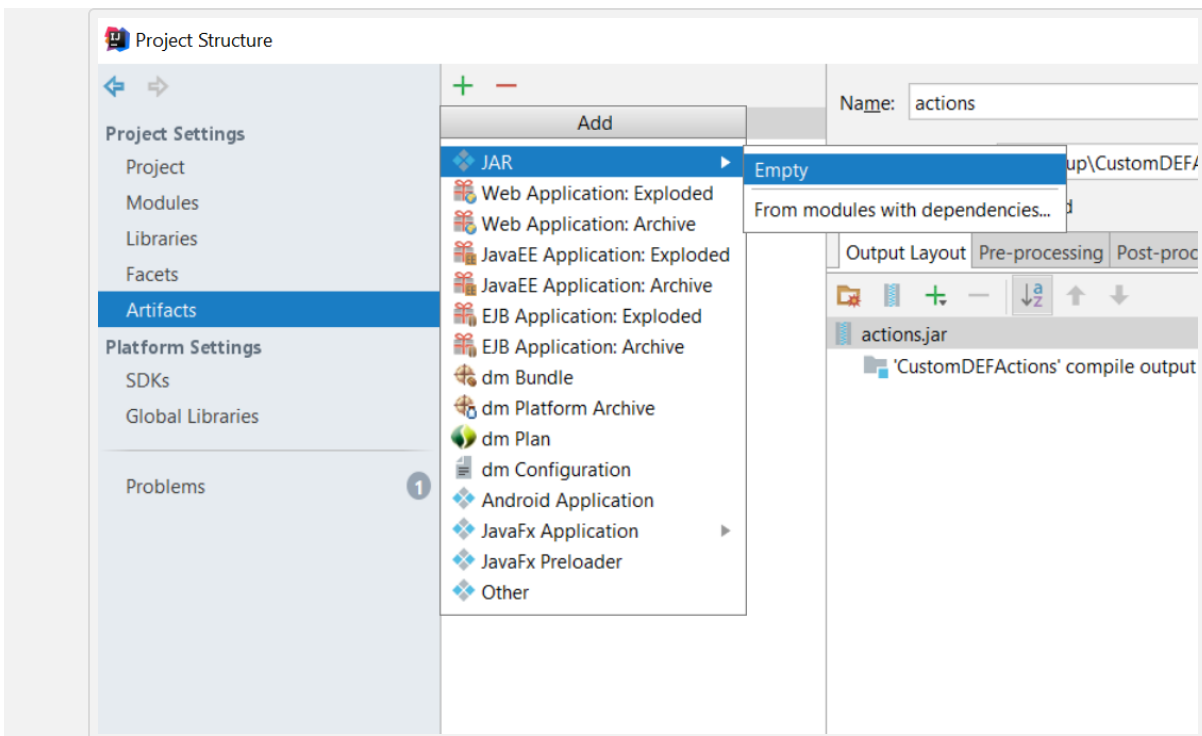
3. Click **Artifacts**.



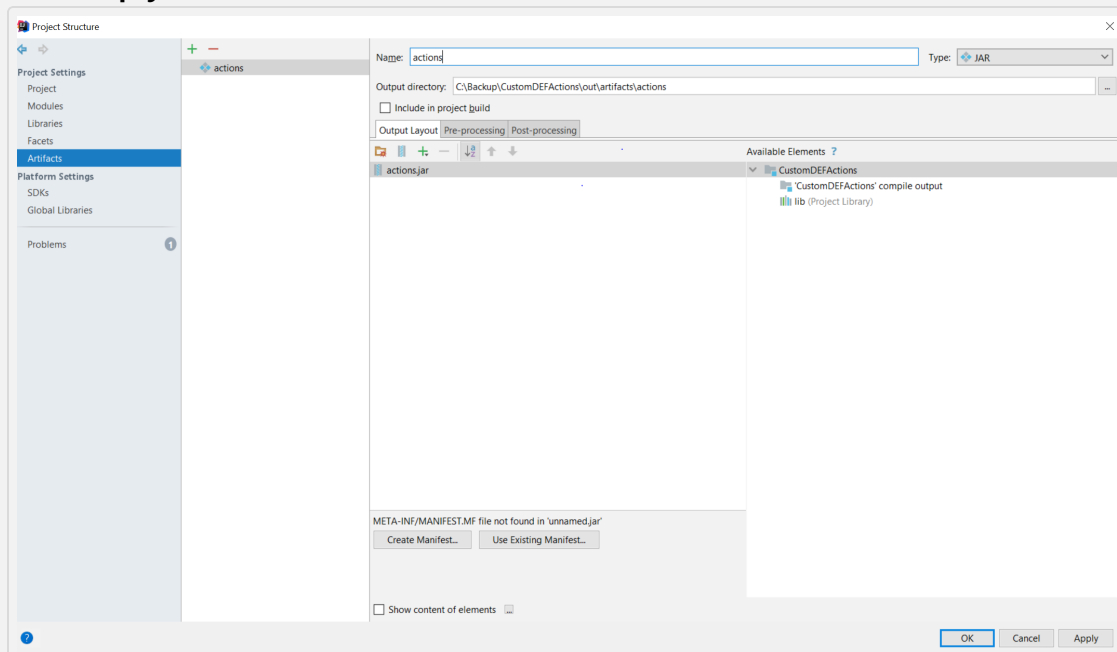
4. Click the plus sign.



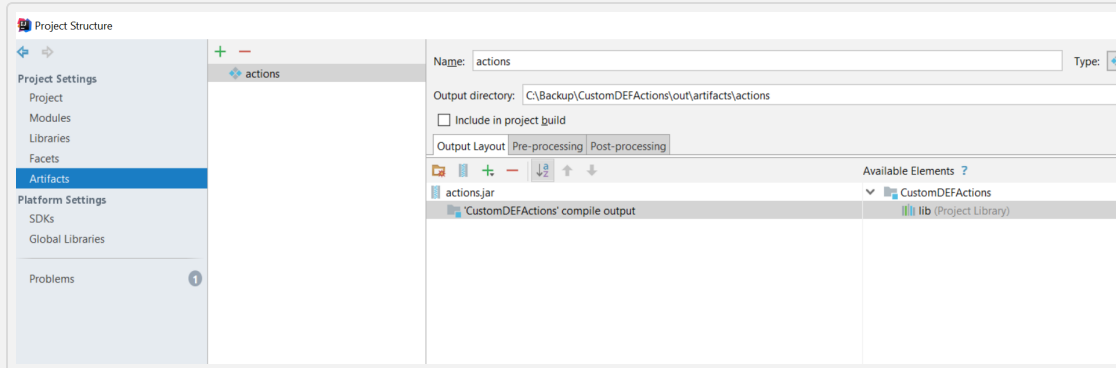
5. Click on **JAR**.



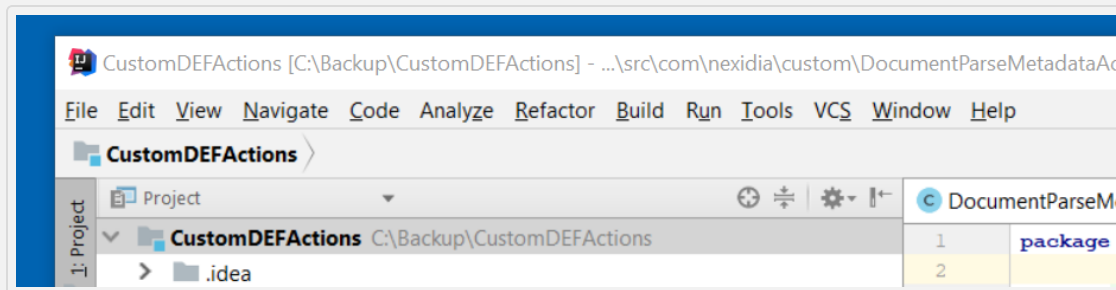
6. Click **Empty**.



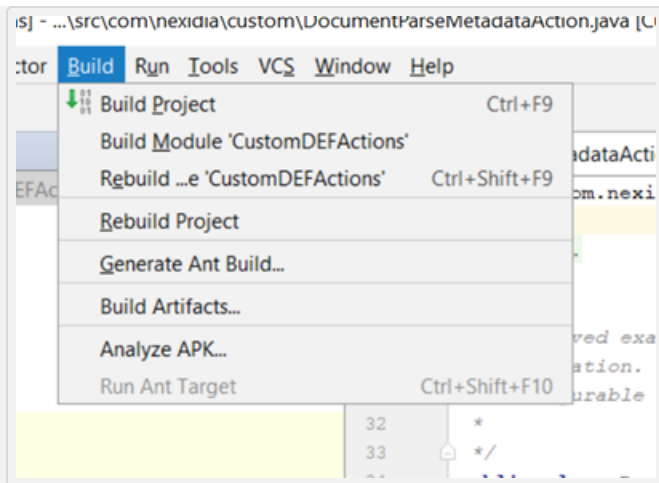
7. Type in **actions** for the name of the JAR file.
8. Click **Available Elements** → <Custom Project Name> → <Custom Project Name> compile output.



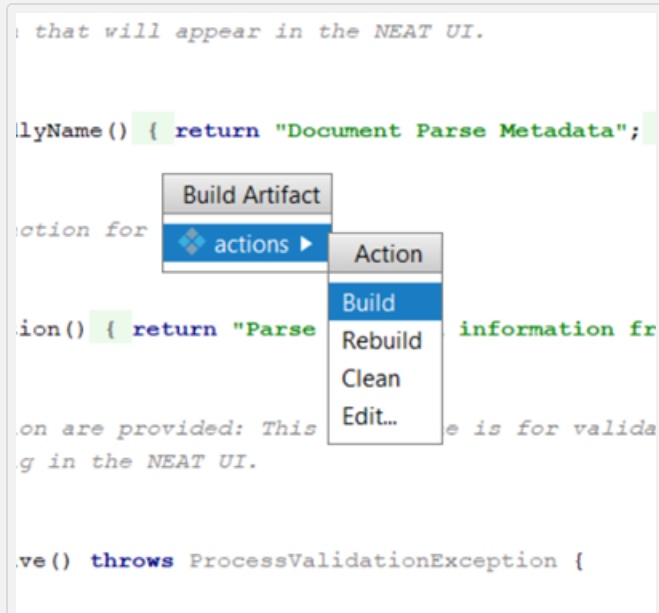
9. Click **OK** to save the JAR file settings.
10. Click the **Build** menu located on the menu bar.



11. Click **Build Artifacts**.



12. Click **Build** to build the actions.jar file.



RESTful API Integrations

This type of integration has become a common use case for custom actions in DEF, since many customers make their metadata and media available behind proprietary RESTful APIs. There is no generic action to make this possible, given the very different implementations encountered, but the general idea should follow a few basic principles:

- The general choice should be an extension of the `ActionBase` abstraction from the SDK, configured to use no inputs and a file system output.
- The quantum is an important aspect of most of these types of integrations, especially regarding metadata retrieval, since RESTful APIs are introducing metadata into DEF. Those can be a bit tricky in the `ActionBase` context, but the basic idea is to:
 - In `getActionConfiguration()`, show that you'll be rolling your own quantum by setting `config.setUsingQuantum(false)`;
 - At the start of the `execute()` method, use this code to create a quantum window:

```
// get the quantum
String quantumWindowPreferenceName = "system-engine-" + this.getClass().getSimpleName() + "-" + processId
+ "-" + processPropertyId + "-";
QuantumWindow quantumWindow = ProcessScheduler.getQuantumWindow(processId, quantumWindowPreferenceName,
LAST_EXECUTION_TIME_PREFERENCE_NAME);
// capture the quantum window in case of failure
quantumPreferenceToWindowMap.put(quantumWindowPreferenceName, quantumWindow);
```

- Use that **QuantumWindow's** start and end times in your API calls for retrieving metadata in a certain date range.
- In a **catch** block wrapping all relevant calls in your `execute()` method, call `cleanupAfterFailure()`, which will reset the quantum if any exceptions were caught.

- When the integration requires pulling both metadata and media, the best practice is to have the metadata pulled first in one action (as above, single threaded in **ActionBase**, using a quantum), and the media pulled in a second one. The second action can be a more straightforward extension of the `ActionBaseFileMover` abstraction in the SDK, and it would take the `FileName` keys from the metadata retriever, use them to pull media, and update the `FileName` accordingly (if necessary).
- There are several reasons for the split into two actions:
 - We built the Data Exchange Framework on a retry mechanism; so if the metadata retrieval is successful, but the media fails for whatever reason (maybe not immediately available), the media pull will continue to be retried on subsequent runs of the process, up to the maximum self-reconciliation days defined at the process level, at which point it will give up and roll them over to the rollover table with an `NxQuarantined` flag set.
 - Monitoring in the live view is clearer, and more accurately represents the actual number of files pulled.
 - Failures are harder to recover from if metadata and media are pulled in a single action.
- Take care to make sure that the authentication token won't expire for the maximum possible duration of the action run.
- If you need an external Java library to support the code, deploy it into the `{DEF_INSTALL}\ext-lib` folder of the NEST server that will execute the action.

Logging, Debugging and Exception Handling

This chapter provides details about the Data Exchange Framework logging, remote debugging, and exception handling functions.

LOGGING

All action components have access to a “logger” object that can be used to log messages using its `debug(...)`, `info(...)`, `warn(...)`, `error(...)`, and `fatal(...)` methods. Each of these provide a standard String parameter implementation in addition to an overloaded form that also accepts an Exception. In all of these cases, the output is logged based on the server’s current debug settings as defined in the Server configuration through the Nexidia Data Exchange Framework Admin Tool user interface.

If the server is configured at an appropriate log level, output goes to the `nx_extractor.log` file in the “logs” folder in the Data Exchange Framework data directory. Additionally, any logs calls to `error(...)` or `fatal(...)` will count toward the action’s failure threshold, if one is configured on the action through the Nexidia Data Exchange Framework Admin Tool user interface, and the act of logging the final error that exceeds that threshold triggers the process to shut down and send notifications out.

The `error(...)` and `fatal(...)` methods additionally provide an overloaded form that accepts an additional string, which is the working table key corresponding to the failing record. When that key is present in the log message, the message (plus the stack trace, if an exception is provided) is recorded in the working table in an **NxFailureMessage** column for the failing record.

The first # errors (configurable at the Server level in the Nexidia Data Exchange Framework Admin Tool user interface.) are shipped from the Nexidia Data Exchange Framework Server Tool server back to the Nexidia Data Exchange Framework Admin Tool server for display in the process builder user interface “View Logs” dialog box for each action.

PERFORM REMOTE DEBUGGING

You can more easily debug and troubleshoot custom actions during development by enabling remote debugging.

Important

Debugging is not recommended in production systems because it could cause a very large performance hit in many scenarios.

→ To configure a Data Exchange Framework installation for remote debugging:

1. Shut down the Data Exchange Framework, open the {DEF_INSTALL_FOLDER}\bin\launch.bat (Windows) or {DEF_INSTALL_FOLDER}/bin/launch.sh (Linux) file in a text editor, and add the following options to the java command alongside the -Xms and -Xmx options (optionally changing the default listening port of 8000):
-Xdebug -agentlib:jdwp=transport=dt_socket,address=8000,server=y,suspend=n
The final command should look something like this:
`jre/bin/java -cp "etc:lib/*" -Xms1024M -Xmx8192M -Xdebug -agentlib:jdwp=transport=dt_socket,address=8000,server=y,suspend=n com.nexidia.neat.init.NDEFStartup`
2. Start the Data Exchange Framework back up, and the server listens on port 8000.
3. Confirm your ability to connect to that server on port 8000 from your development environment.
4. Configure Eclipse for remote debugging by going to Run—>Debug Configurations—>Remote Java Application, and clicking the “New Launch Configuration” icon in the upper left of the “Debug Configurations” window. Name the configuration as you like, tie it to your CustomDEFActions project with a “Standard (Socket Attach)” connection type, and enter your host address and port (default=8000).
Once applied and connected, normal debugging is available through Eclipse.

EXCEPTION HANDLING

A `ProcessValidationException` should be thrown by actions during action configuration saves or process publishes, and not during execution itself. These exceptions will bubble up back to the user in the user interface, and should show what’s misconfigured.

Starting with the `ActionBase` abstraction, and down into its extensions (for example: `ActionBaseFileProcessor`), these are thrown from within the `validateSave(...)` method, which is called when an action's configuration is changed and the "Save" button is processed, and the `validateRuntime(...)` method, which is called when a process is published. `ActionComponent` extensions should validate in the `execute(...)` method, using the "validateOnly" parameter to decide on what should be executed fully, or what should/can only be validated.

A `ProcessExecutionException` is a far more serious exception in the Data Exchange Framework process engine, and should only be thrown when your action has hit an error that it can't recover from. Usually this will be something unexpected, for example in the structure of an input file, or in establishing a connection to an outside API.

In most cases, using the `logger.error(...)` methods will be most appropriate for custom actions using the `ActionBase` abstraction and its extensions.